Guide d'Étude Structuré : Architecture et Modélisation des Bases de Données

Niveau 1 : Concepts Fondamentaux et Fondation des Données

Avant de manipuler un système technique, il convient de comprendre la nature de l'information et les outils fondamentaux de sa gestion.

- **`Information structure` (Data, Information, Knowledge, Wisdom)** : Cette pyramide théorique définit le continuum de l'abstraction. Les signaux bruts (`Data`) sont contextualisés pour devenir de l'information, synthétisés en règles (`Knowledge`), puis appliqués avec jugement (`Wisdom`). Les bases de données gèrent principalement les couches `Data` et informationnelles.
- **Cycles de vie de la donnée** :
  - **`Persistent Data`** : Une information stockée sur une mémoire non volatile (disque dur). Elle survit à l'arrêt du processus informatique et constitue l'état fondamental du système.
  - **`Transient Data`** : Une information éphémère maintenue exclusivement dans la mémoire volatile (`Random Access Memory`) durant un calcul, vouée à être détruite à l'arrêt du processus.
- **`Database`** : Une collection structurée, organisée et persistante de `Data`, conçue pour permettre une interrogation et une manipulation extrêmement efficaces. C'est le conteneur passif de l'information.
- **`Database Management System` (DBMS / SGBD)** : L'infrastructure logicielle complexe et active qui sert d'interface entre les utilisateurs, les applications et le `Database`. Il orchestre le stockage, gère la concurrence et garantit la sécurité des opérations.
- **`Metadata`** : Une donnée auto-descriptive qualifiant la structure, la nature ou la sémantique d'autres `Data` (l'information décrivant le contenant plutôt que le contenu).
- **Référentiels et Gouvernance** :
  - **`Data Dictionary`** : Le composant interne du `DBMS` qui stocke la définition formelle de toutes les structures (`Table`, `View`, `Index`) et règles de sécurité.
  - **`Data Catalog`** : Un inventaire organisationnel de plus haut niveau utilisant les `Metadata` pour faciliter la découverte et la gouvernance de l'information au sein d'une entreprise.
- **`Data Redundancy`** : Une anomalie architecturale consistant à dupliquer inutilement la même information persistante en de multiples endroits, ce qui gaspille de l'espace et génère un risque majeur de contradictions.

Niveau 2 : Architecture des Bases de Données et Cycle de Vie

Les étapes de conception d'un système et ses différents niveaux d'abstraction architecturale.

- **`Database Lifecycle`** : Le cadre méthodologique régissant la conception d'un système d'information. Il se compose de plusieurs phases séquentielles : `Analysis`, `Modeling`, `Implementation`, `Test/Evaluation` et `Use`.
- **`Data Independence`** : Le principe d'ingénierie fondamental visant à séparer la définition structurelle des `Data` de la logique des applications métier, permettant ainsi de modifier le stockage sans casser les programmes.
- **L'architecture à trois niveaux (ANSI-SPARC)** :

- `External Schema` **(Couche externe)** : Le niveau d'abstraction le plus haut, présentant des vues sur mesure et sécurisées adaptées à chaque utilisateur ou application spécifique, masquant la complexité technique.
- `Logical Schema` / `Community Schema` **(Couche conceptuelle et logique)** : Le plan technique partagé de toute l'organisation. Il traduit les concepts métier en une structure mathématique rigoureuse, indépendamment de toute considération matérielle.
- `Physical Schema` / `Internal Schema` **(Couche physique)** : La configuration technique ultime implémentée sur un `DBMS` spécifique. Il dicte l'allocation spatiale sur les disques durs, le partitionnement et l'indexation pour optimiser les performances.

Niveau 3 : Modélisation Conceptuelle et Entité-Association

La planification de la structure de la base de données via des modèles abstraits avant l'écriture du code.

- `Data Modeling` : Le processus intellectuel d'abstraction d'un système d'information en schémas formels afin de garantir la normalisation des `Data`.

- `Entity-Relationship Model` : Un métamodèle sémantique visuel utilisé pour cartographier les objets métier et leurs interactions sans se préoccuper de la technologie de stockage.

- **Entités et Attributs** :

  - `Entity Class` : Le modèle abstrait (le moule) définissant une catégorie d'objets partageant la même structure. Une `Strong Entity Class` possède sa propre identité autonome, tandis qu'une `Weak Entity Class` dépend de l'existence d'une classe parente pour être identifiée.
  - `Entity` : Une instance unique et distincte appartenant à une `Entity Class` (ex: un client précis).
  - `Attribute` : Une propriété définissant une caractéristique spécifique. Il peut être `Atomic` (indivisible), `Composite` (composé de sous-attributs), `Derived` (calculé dynamiquement sans être stocké physiquement) ou `Identifying` (permettant de distinguer une `Entity` de manière univoque).

- **Relations et Topologie** :

  - `Association` : Le lien sémantique et structurel établi entre différentes `Entity Class`. Une association peut être `Binary` (deux classes), `Ternary` (trois classes indissociables), `Cyclic` (une boucle de dépendance), ou `Recursive Association` (une classe liée à elle-même, requérant des `Roles in Association` pour clarifier l'interaction).

```sql
CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    ManagerID INT,
    FOREIGN KEY (ManagerID) REFERENCES Employee(EmployeeID)
);
```

  - `Association Cardinality` (`One-to-One`, `One-to-Many`, `Many-to-Many`) : Les limites mathématiques qui définissent le nombre minimum et maximum d'instances pouvant participer à une `Association`.

Niveau 4 : Modélisation Avancée et Concepts Orientés Objet

La gestion des hiérarchies complexes et l'intégration des paradigmes de la programmation moderne.

- **`Object-Oriented Model`** : Un paradigme où les `Data` sont encapsulées avec leurs comportements algorithmiques au sein de classes complexes, par opposition aux grilles relationnelles classiques.
- **`UML Class Diagram`** : Une notation visuelle standardisée du langage `UML` permettant de modéliser l'architecture statique d'un système, servant souvent de plan pour générer du code applicatif et les scripts de la base de données.
- **Héritage et Hiérarchies** :
  - **`Specialization-Generalization`** : L'axe d'abstraction consistant soit à détailler une `Superclass` en plusieurs sous-groupes spécifiques, soit à regrouper des caractéristiques communes vers le haut.
  - **`Specialization Cardinality`** : Les règles définissant si une sous-catégorisation est obligatoire (`Total`) ou optionnelle (`Partial`), et si elle est disjointe (`Exclusive`) ou superposée (`Overlapping`).
  - **`Is-A Relationship`** : Le principe sémantique asymétrique affirmant qu'une `Subclass` est une occurrence parfaitement valide de sa `Superclass`.
  - **`Inheritance`** : Le mécanisme par lequel une `Subclass` dérive automatiquement des `Attribute` de sa `Superclass`. On parle de **`Multiple Inheritance`** ou de **`Multiple Specialization`** lorsque les dérivations proviennent de plusieurs classes ou axes orthogonaux, ce qui exige des stratégies d'aplatissement complexes dans un système relationnel.

Niveau 5 : Le Modèle Relationnel et l'Intégrité des Données

La traduction des concepts abstraits en tableaux mathématiques strictement régulés.

- **`Relational Model`** : Le paradigme mathématique standard structurant la totalité des `Data` sous forme d'enregistrements (`Record`) au sein de relations bidimensionnelles plates (`Table`).

- **`Relational Schema`** : Le plan technique formel issu du modèle relationnel, définissant mathématiquement l'architecture des `Table`, de leurs `Column` et des règles appliquées.

- **Identification et Connexion** :

  - **`Primary Key (PK)`** : Un ou plusieurs `Attribute` garantissant l'identité unique, exclusive et non vide de chaque `Record` au sein d'une `Table`.
  - **`Foreign Key (FK)`** : Une référence sécurisée pointant vers la `Primary Key` d'une autre (ou de la même) `Table`. Elle sert à maintenir les listes connectées.

  ```sql
  ALTER TABLE ChildTable
  ADD CONSTRAINT FK_Parent
  FOREIGN KEY (ParentID) REFERENCES ParentTable(PrimaryID);
  ```

- **Garantie de la Cohérence** :

  - **`Null Value`** : Un marqueur sémantique spécial indiquant l'absence d'information, son statut inconnu ou son inapplicabilité (différent de zéro ou d'un texte vide).
  - **`Integrity Constraint`** : Des règles de sécurité déclaratives strictes empêchant l'insertion de données invalides (vérification de domaine, contraintes de lignes ou de colonnes).
  - **`Referential Integrity`** : Le mécanisme garantissant qu'une `Foreign Key` ne pointe jamais vers une `Primary Key` inexistante, empêchant la création de données orphelines.

- **`View`** : Une relation virtuelle instanciée dynamiquement par l'exécution d'une requête persistante. Elle n'enregistre aucune donnée physique mais simplifie l'accès et sécurise l'information.

```sql
CREATE VIEW ActiveUsers AS
SELECT UserID, Username
FROM User
WHERE IsActive = 1;
```

Niveau 6 : Interrogation et Langages de Communication

Les moyens syntaxiques employés pour interagir avec l'infrastructure du système de gestion.

- **Paradigmes de Langage** :
  - **`Declarative Language`** : Un langage où l'utilisateur spécifie uniquement le résultat attendu ("quoi") sans programmer explicitement l'algorithme d'exécution séquentielle ("comment"). Le `Query Optimizer` du système décide du chemin d'accès optimal.
  - **`Procedural Language`** : Un langage de programmation impératif exigeant de définir la séquence exacte d'instructions, de boucles et de branchements (souvent utilisé pour les `Stored Procedure` et les `Trigger`).

```sql
CREATE PROCEDURE UpdateSalaries()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    -- Boucles et curseurs procéduraux omis pour concision
    UPDATE Employee SET Salary = Salary * 1.05;
END;
```

- **`Structured Query Language` (SQL)** : Le langage déclaratif standardisé universel conçu pour la définition, la manipulation et l'interrogation au sein des `DBMS` relationnels.

- **Sous-ensembles de commandes SQL** :
  - **`Data Definition Language` (DDL)** : Les commandes (ex: `CREATE`, `ALTER`) utilisées exclusivement pour bâtir ou modifier la structure et les `Metadata` du `Database Schema`, sans toucher aux enregistrements.

```sql
CREATE TABLE Product (
    ProductID INT PRIMARY KEY,
```

```
    Name VARCHAR(100) NOT NULL
);
```

- **`Data Manipulation Language` (DML)** : L'interface d'écriture transactionnelle permettant d'insérer, de mettre à jour ou de supprimer les `Data` opérationnelles (`INSERT`, `UPDATE`, `DELETE`).
- **`Data Retrieval Language` (DRL)** : La composante spécialisée dans le filtrage et l'extraction d'information sans altération de l'état persistant de la base (`SELECT`).

```sql
SELECT Name, Price
FROM Product
WHERE Price > 100;
```

Niveau 7 : Transactions et Concurrence

La préservation de l'exactitude des données dans des environnements temps réel multi-utilisateurs.

- **`Concurrent Access`** : La situation normale mais complexe où de multiples processus tentent de lire ou de modifier le même sous-ensemble de `Data` simultanément.
- **`Transaction`** : Une unité logique de traitement regroupant une ou plusieurs instructions `SQL` en un bloc indivisible. Elle garantit le passage d'un état de cohérence validé à un autre.
- **Le Paradigme `ACID`** : Un contrat de fiabilité absolu appliqué par le `DBMS` sur les transactions :
  - `Atomicity` (L'opération réussit entièrement ou échoue entièrement).
  - `Consistency` (Le passage d'un état valide à un autre état valide est garanti).
  - `Isolation` (La protection contre les effets de bord liés au `Concurrent Access`).
  - `Durability` (La pérennité absolue de la modification une fois validée).
- **`Rollback`** : L'opération d'urgence déclenchée face à une anomalie. Elle efface immédiatement toutes les modifications `DML` en cours et restaure le `Database` à son état de consistance précédent.

```sql
BEGIN TRANSACTION;
DELETE FROM ImportantTable WHERE ID = 42;
-- Une erreur critique est détectée
ROLLBACK TRANSACTION;
```

Niveau 8 : Histoire et Outillage Analytique

Le contexte historique et les logiciels d'ingénierie employés dans le domaine.

- **`CODASYL`** : Un standard historique définissant le `Network Model` (Modèle Réseau). Précurseur du `Relational Model`, il organisait l'information avec des pointeurs physiques complexes, rendant l'interrogation fortement dépendante du stockage sur disque.

- **`DB-Main`** : Un outil d'ingénierie et de conception spécialisé. Il est utilisé lors des phases analytiques pour créer les modélisations, effectuer de la rétro-ingénierie et générer automatiquement le `Logical Schema` puis le `Physical Schema` à partir d'un `Conceptual Schema`.